

Báo cáo dự án — Chess AI (AlphaZero-lite)

Môn AIT2004 — Cơ sở AI. Bot cờ vua kết hợp mạng neural (policy + value), search cổ điển (Minimax/MCTS) và vòng lặp self-play kiểu AlphaZero thu nhỏ.

1. Tóm tắt

Dự án xây dựng một engine cờ vua học từ dữ liệu chuyên gia rồi tự cải thiện qua self-play. Pipeline gồm 5 khối: parse PGN → encode bàn cờ → mạng CNN hai đầu (policy + value) → search (Minimax alpha-beta hoặc MCTS) → vòng lặp self-play (sinh data → retrain → pit → giữ model mạnh hơn).

Kết quả chính: sau khi train supervised trên **1,77 triệu thế cờ** từ 20.000 ván Lichess Elite, model đạt **val top-1 = 33,5%** và **top-5 = 66,0%** trong việc dự đoán nước đi của chuyên gia — đúng mức kỳ vọng cho một mạng ~1,43M tham số. Pipeline self-play chạy trơn vẹn; lần chạy đầu gặp vấn đề "toàn hòa", đã được vá bằng adjudication theo material (kèm ELO tracking và batch inference). Sau khi vá, pit luôn phân thắng bại (0 ván hòa) nhưng self-play vẫn **chưa cải thiện** được model — với lượng self-play nhỏ (20 ván/vòng) và learning rate cao, retrain làm hỏng trọng số pretrain. Benchmark với Stockfish depth-2: bot đạt **3,0/10 (30%)** — hòa 6, thua 4, thắng 0. Toàn bộ logic không-cần-GPU được kiểm thử đơn vị (36 test, 7 file).

2. Kiến trúc hệ thống

```
PGN games -> Encode 12x8x8 -> CNN (Policy + Value heads)
                                   |
                                   Minimax/Alpha-Beta hoặc MCTS (PUCT)
                                   |
                                   Self-play -> Retrain -> Pit (lặp)
```

Encode bàn cờ. Mỗi thế cờ → tensor $12 \times 8 \times 8$ one-hot (6 loại quân $\times$ 2 màu). Khi đến lượt Đen, bàn cờ được xoay 180° và đảo màu để mạng luôn "nhìn" mình là Trắng.

Action space. $4096 = 64 \times 64$ (ô đi → ô đến). Đơn giản, phủ hầu hết nước đi; chấp nhận mất thông tin phong cấp dưới hậu (rất hiếm).

Mạng PolicyValueNet. Thân chung: 1 conv stem ($12 \rightarrow 128$) + 3 residual block (128 kênh). Hai đầu ra: *policy head* (→ 4096 logits) và *value head* (→ $\tanh \in [-1, 1]$). Chi tiết ở Mục 2.2.

Search. Minimax + Alpha-Beta (negamax, eval tĩnh, không cần value head) hoặc MCTS PUCT (dựa vào policy + value của mạng). Chi tiết ở Mục 5.

Opening book. `src/search/opening_book.py` hardcoded ~15 khai cuộc phổ biến; khi bật, bot đi theo sách đầu ván rồi mới search.

2.1. Cấu trúc mã nguồn (module-by-module)

File	Vai trò
src/data/encode_board.py	Board → tensor 12×8×8 one-hot + perspective flip cho Đen-to-move.
src/data/action_space.py	Move ↔ index (4096 = from×64+to); mask nước hợp lệ.
src/data/pgn_parser.py	Đọc PGN/pgn.zst → mẫu (fen, move, result) lưu .pt.
src/data/dataset.py	PyTorch Dataset + DataLoader, split train/val 90/10.
src/model/network.py	PolicyValueNet (trunk + policy head + value head) và PolicyNet.
src/search/evaluation.py	Eval tĩnh material+PST; adjudicate_result (chấm material khi cắt ván).
src/search/minimax.py	Negamax + alpha-beta; move ordering bằng policy net.
src/search/mcts.py	MCTS PUCT; batch inference (num_parallel) + virtual loss.
src/search/opening_book.py	~15 khai cuộc hardcoded; book_move(board) .
src/search/training/supervised.py	Train supervised trên PGN (CE policy + MSE value).
src/search/training/self_play.py	Sinh self-play games bằng MCTS.
src/search/training/pit.py	Đấu 2 model, chấm thắng/hòa (có adjudication).
src/search/training/self_play_loop.py	Vòng lặp generate→retrain→pit→giữ; log ELO.
src/search/training/elo.py	Tính ELO tương đối từ kết quả pit.
app.py	Web app Flask (chơi với bot trong trình duyệt).
scripts/*.py	Entry-point: parse_pgn, train_supervised, run_self_play, play_cli, benchmark_stockfish, plot_history.
tests/*.py	Unit test (pytest) — xem Mục 8.

Lưu ý cấu trúc: các module training nằm trong `src/search/training/` (không phải `src/training/` như mô tả cũ trong README gốc).

2.2. Chi tiết kiến trúc mạng (layer-by-layer)

Luồng tensor qua mạng (batch kích thước B):

```

input                                (B, 12, 8, 8)
stem : Conv2d(12->128, 3x3, pad 1, bias=False) +BN +ReLU    -> (B, 128, 8, 8)
res×3 : [ Conv(128->128,3x3)+BN+ReLU -> Conv(128->128,3x3)+BN ]
        + skip connection, rồi ReLU                            -> (B, 128, 8, 8)
-- policy head --
  Conv2d(128->2, 1x1, bias=False) +BN +ReLU                  -> (B, 2, 8, 8)
  flatten                                                       -> (B, 128)
  Linear(128 -> 4096)                                           -> (B, 4096)   = policy logits
-- value head --
  Conv2d(128->1, 1x1, bias=False) +BN +ReLU                  -> (B, 1, 8, 8)
  flatten                                                       -> (B, 64)
  Linear(64 -> 64) +ReLU                                         -> (B, 64)
  Linear(64 -> 1) + tanh                                         -> (B,)       = value in [-1, 1]

```

Bảng đếm tham số (tính chính xác từ kiến trúc 128 kênh / 3 block):

Khối	Chi tiết	Tham số
Stem (conv + BN)	$12 \cdot 128 \cdot 3 \cdot 3 + 128 \cdot 2$	14.080
3 × Residual block	mỗi block 2 conv 3×3 (147.456/conv) + 2 BN	886.272
Policy head	conv 1×1 (256) + BN (4) + FC $128 \rightarrow 4096$ (528.384)	528.644
Value head	conv 1×1 (128) + BN (2) + FC $64 \rightarrow 64$ (4.160) + FC $64 \rightarrow 1$ (65)	4.355
Tổng		1.433.351 (~1,43M)

Nhận xét: ~62% tham số nằm ở 3 residual block (phần "suy nghĩ"), ~37% nằm riêng ở lớp `Linear(128→4096)` của policy head (do output 4096 lớn). **Value head cực nhẹ (~4.355 tham số, ~0,3%)** — vừa ít dung lượng vừa làm nhiệm vụ khó (đoán kết quả từ một thế cờ), một lý do cấu trúc khiến nó là phần yếu nhất.

2.3. Encode bàn cờ & quy ước nhãn (làm rõ các điểm dễ nhầm)

- **12 kênh one-hot:** 6 loại quân $\times$ 2 màu. Kênh `c`, `ô` (`rank`, `file`) = 1 nếu ô đó có quân loại `c`. Tổng tensor = số quân trên bàn (32 ở thế đầu) — đã kiểm tra trên dữ liệu thật.
- **Perspective flip:** khi tới lượt Đen, bàn cờ được xoay 180° và đảo kênh màu Trắng↔Đen, để mạng **luôn "thấy" mình là bên Trắng đi từ dưới lên**. Mạng không phải học riêng cho Trắng/Đen → tận dụng đối xứng, ít dữ liệu vẫn học được.
- **Policy target (supervised):** là **chỉ số nước đi của kỳ thủ chuyên gia** trong ván thật (1 trong 4096 lớp). Loss policy = cross-entropy như phân loại 4096 lớp → mạng học "ở thế này nên đi nước nào". Công thức index: `from_square·64 + to_square`.
- **Value target (supervised):** kết quả ván, **đổi dấu theo bên đang đi**: +1 nếu bên-đến-lượt thắng ván đó, −1 nếu thua, 0 nếu hòa. Loss value = MSE giữa output `tanh` và target.
- **top-1 / top-5 accuracy:** top-1 = nước mạng cho xác suất cao nhất có trùng đúng nước chuyên gia không; top-5 = nước chuyên gia có nằm trong 5 nước mạng ưu tiên nhất không. Đây là thước đo "bắt chước chuyên gia giỏi đến đâu", không phải sức cờ tuyệt đối.

3. Dữ liệu

Thuộc tính	Giá trị
Nguồn	Lichess Elite 2024-01 (298k ván, rating $\geq$ ~2400)
Lọc	<code>min_rating 1800</code> , <code>min_moves 10</code>
Số ván dùng	20.000
Số thế cờ (mẫu)	1.768.985
Cân bằng nhãn	Trắng thắng 37,5% · Đen thắng 37,1% · Hòa 25,4%

Pipeline dữ liệu: `parse_games()` duyệt từng ván, lọc theo rating/độ dài, với mỗi nước đi sinh một mẫu (`fen` trước nước đi, `uci`, kết quả ván từ góc Trắng). Lưu `.pt`. Hỗ trợ cả `.pgn` và `.pgn.zst` (nén). Nhãn rất cân bằng → không lệch về một phía, tốt cho value head. Kiểm tra encode: mọi thế đầu ván có tổng = 32 quân, tensor chỉ chứa 0/1 (one-hot sạch); 100% chỉ số nước đi nằm trong `[0, 4095]`.

4. Pipeline huấn luyện & kết quả supervised

4.1. Pipeline huấn luyện (chi tiết từng bước)

1. **Dữ liệu → tensor.** ChessDataset đọc mỗi fen, encode perspective **on-the-fly** (không lưu sẵn tensor → file nhỏ), tính `policy = uci_to_index(move)`, `value = result` đổi dấu theo bên đi. Split train/val **90/10** (seed cố định 42 → tái lập được).
2. **Hàm mất mát (loss).** `L = CrossEntropy(policy_logits, expert_move) + value_weight × MSE(value_pred, value_target)`, với `value_weight = 1.0`.
3. **Optimizer.** AdamW, `lr = 1e-3`, `weight_decay = 1e-4`.
4. **Lịch học (LR schedule).** CosineAnnealingLR với `T_max = epochs × số_batch`, **bước mỗi batch** → lr giảm mượt từ 1e-3 về 0 (xem cột `lr` ở log).
5. **Gradient clipping.** `clip_grad_norm(max_norm = 1.0)` mỗi bước → chống nổ gradient.
6. **Batch / epoch.** Batch 256; mỗi epoch chạy hết tập train rồi đánh giá trên tập val.
7. **Checkpoint.** `latest.pt` lưu sau **mỗi** epoch; `best.pt` lưu **khi val top-1 đạt kỷ lục mới** → tự giữ epoch khá nhất.
8. **Metrics log mỗi epoch.** policy loss, value loss, top-1, top-5 (train & val) + value MAE (val).

4.2. Minh chứng — log huấn luyện đầy đủ (10 epoch)

Cấu hình thực tế (đọc từ `best.pt`): `channels=128, n_res_blocks=3, epochs=10, batch_size=256, lr=1e-3, weight_decay=1e-4, value_weight=1.0, val_split=0.1, device=cpu`. Dữ liệu: 20.000 ván → **1.768.985 thế cờ**.

epoch	lr	train p-loss	train v-loss	train top1	train top5	val p-loss	val v-loss	val top1	val top5	val MAE	thời gian
1	9,76e-4	3,821	0,721	20,3%	45,1%	3,169	0,720	24,9%	54,4%	0,750	62 ph
2	9,05e-4	2,968	0,694	27,5%	58,3%	2,933	0,720	28,1%	59,4%	0,741	55 ph
3	7,94e-4	2,741	0,674	30,7%	63,1%	2,799	0,678	30,0%	61,9%	0,720	52 ph
4	6,55e-4	2,591	0,651	33,0%	66,3%	2,727	0,661	31,1%	63,6%	0,703	61 ph
5	5,00e-4	2,470	0,624	35,0%	68,9%	2,674	0,653	32,2%	64,8%	0,692	80 ph
6	3,45e-4	2,364	0,594	36,9%	71,1%	2,647	0,624	32,6%	65,4%	0,669	78 ph
7	2,06e-4	2,271	0,564	38,8%	73,0%	2,635	0,626	33,0%	65,7%	0,657	78 ph
8	9,55e-5	2,193	0,536	40,4%	74,5%	2,634	0,608	33,3%	66,1%	0,642	223 ph*
9	2,45e-5	2,135	0,515	41,7%	75,6%	2,639	0,608	33,5%	66,0%	0,635	62 ph
10	0	2,103	0,503	42,4%	76,3%	2,642	0,610	33,5%	66,0%	0,633	64 ph

Train trên **CPU**, tổng ~**13,6 giờ**. (*Epoch 8 mất ~3,7h là đột biến do máy bận, không phải do thuật toán — các epoch khác ~1h.) `best.pt` lưu ở epoch có val top-1 cao nhất (epoch 9-10, val top-1 = 0,3349).

4.3. Đọc log: overfitting, lịch học, value head

Overfitting (policy). Train top-1 tăng đều suốt 10 epoch (20,3% → 42,4%), nhưng **val top-1** **chững lại ~33,5% từ epoch 7-8**. Khoảng cách train/val nới rộng dần (epoch 10:

train p-loss 2,10 vs val 2,64). Val policy loss **đạt đáy ở epoch 8 (2,6335)** rồi nhích lên → overfit bắt đầu từ epoch ~7-8. Dừng ở 10 epoch là hợp lý.

Quan hệ policy ↔ value (làm rõ theo góp ý review). Không nên chỉ gọi đây là "overfit" chung chung. Nhìn riêng các epoch cuối: từ epoch 8→10 val policy loss *tăng* (2,634 → 2,642) **trong khi** val value MAE *vẫn giảm đều* (0,642 → 0,635 → 0,633). Vì policy và value **dùng chung trunk** và loss là tổng có trọng số, mẫu hình này gợi ý trunk đang dần dần nâng lực về value và **đánh đổi khả năng khái quát của policy** — chứ không thuần túy là overfit cổ điển. Tuy nhiên độ lớn nhỏ (value chỉ nhỉnh ~0,01 MAE/2 epoch, val value loss gần như phẳng 0,608→0,610), nên đây mới là **giả thuyết trade-off phù hợp với dữ liệu, chưa phải kết luận**. Muốn khẳng định cần **ablation**: train một bản `value_weight=0` (policy-only) hoặc quét `value_weight ∈ {0.5, 1, 2}` rồi so đường val policy giữa các cấu hình.

Value head — vì sao gọi là "yếu". Vì 74,6% ván phân thắng bại ($|value| = 1$) và 25,4% hòa ($value = 0$), một bộ "luôn đoán hòa (0)" có MAE ≈ **0,746**. Val MAE của mạng đi từ 0,750 (epoch 1, ~bằng đoán-hòa) xuống **0,633** → value head **có học** nhưng chỉ nhỉnh hơn baseline "luôn hòa" một khoảng khiêm tốn (~0,11). Đây là minh chứng định lượng cho điểm yếu cốt lõi, lý giải vì sao MCTS chưa mạnh và bot hòa nhiều nhưng không biết thắng (Mục 7).

Đối chiếu biểu đồ: `reports/supervised.png` — 3 panel: policy loss (train tách dần khỏi val), value head (loss/MAE giảm rồi phẳng), accuracy (val bão hòa trong khi train tiếp tục lên).

4.4. Ảnh hưởng kích thước dữ liệu

	2.000 ván (5 epoch)	20.000 ván (10 epoch)
Số thể cờ	178.362	1.768.985
val top-1	20,8%	33,5%
val top-5	44,4%	66,0%

Tăng dữ liệu ~10× đẩy top-1 từ 20,8% lên 33,5% và lùi điểm overfit ra xa hơn — minh chứng **dữ liệu là yếu tố quyết định** chứ không phải số epoch. Ý nghĩa con số cuối: top-1 33,5% = cứ 3 thể cờ đoán đúng 1 nước y hệt chuyên gia; top-5 66% = nước đúng nằm trong 5 gợi ý đầu ở 2/3 số thể.

5. Thuật toán search

5.1. Hàm đánh giá tĩnh (cho Minimax)

`evaluate_board()` chấm điểm centipawn từ góc nhìn Trắng = **material + piece-square table (PST)**.

- **Material:** Tốt 100, Mã 320, Tượng 330, Xe 500, Hậu 900, Vua 20000.
- **PST:** mỗi loại quân có bảng 64 ô thưởng/phạt theo vị trí (vd Mã ở trung tâm > Mã ở góc; Tốt càng tiến càng cao điểm). Quân Đen dùng bảng lật theo rank.
- **Thế kết thúc:** chiếu hết ±100000; hòa (stalemate / thiếu quân / 50 nước / lặp 3 lần) = 0.

5.2. Minimax + Alpha-Beta

- **Negamax:** tận dụng tính zero-sum — `score = -negamax(con)` ở mỗi tầng, gọn hơn viết max/min riêng.
- **Alpha-Beta pruning:** cắt nhánh khi `alpha ≥ beta` → giảm mạnh số node phải duyệt mà vẫn ra cùng kết quả.
- **Move ordering:** nếu có model, sắp nước theo prior của policy net (nước "hứa hẹn" duyệt trước → cắt nhánh sớm hơn); nếu không, dùng MVV-LVA (ưu tiên ăn quân giá trị cao) + nước chiếu.
- Không dùng value head → chạy được kể cả khi chưa có/không cài torch.

5.3. MCTS (PUCT, kiểu AlphaZero)

Bốn pha mỗi simulation: 1. **Selection:** từ root đi xuống theo công thức **PUCT**: $UCB(s,a) = Q(s,a) + c_{puct} \cdot P(s,a) \cdot \sqrt{N_{parent}} / (1 + N(s,a))$ với Q = giá trị trung bình, P = prior từ policy net, N = số lần thăm, $c_{puct} = 1.5$. (Trong code, Q được đảo dấu theo góc nhìn parent vì hai bên đổi lượt.) 2. **Expansion:** gập leaf chưa mở → đánh giá bằng mạng (policy + value), tạo các con với prior. 3. **Evaluation:** value của leaf lấy từ value head (hoặc $\pm 1/0$ nếu thế cờ kết thúc). 4. **Backup:** lan value ngược lên root, **đổi dấu mỗi tầng** vì side đổi lượt.

Tăng cường: **Dirichlet noise** tại root khi self-play ($\alpha=0.3$, $\epsilon=0.25$) để khám phá; **batch inference** (`num_parallel`) gom nhiều leaf đánh giá 1 lần (virtual loss để các leaf không trùng nhánh) — tăng tốc trên GPU, mặc định 1 = như cũ.

Policy và value tham gia thế nào trong công thức (và vì sao value head là nút thắt). Hai đại lượng của mạng vào thẳng PUCT theo hai vai trò khác nhau: - **P (policy prior)** nằm ở **số hạng khám phá** $c_{puct} \cdot P \cdot \sqrt{N_{parent}} / (1+N)$ — quyết định nhánh nào được ưu tiên *thử trước*. Policy tốt → MCTS không phí simulation vào nước vô lý. - **Q (suy từ value head)** là **số hạng khai thác** — value head đánh giá leaf, giá trị được backup thành Q ở mỗi node, quyết định nước nào đang "tốt".

Do đó chất lượng nước MCTS chọn **phụ thuộc trực tiếp vào độ chính xác của value**: value MAE $\sim 0,63$ ($\approx$ chỉ nhỉnh hơn "đoán hòa", Mục 4.3) → Q ước lượng nhiễu → MCTS chầm sai nhiều thế, nhất là tàn cuộc khi material ngang nhau. Đây chính là **chuỗi nhân quả** dẫn tới kết luận *value head là nút thắt của MCTS* (và lý giải bot hòa được nhưng không thắng được Stockfish, Mục 7) — không phải khẳng định suông. Policy tuy mạnh hơn (top-1 33,5%) nhưng không đủ bù: MCTS vẫn cần Q đáng tin để chọn giữa các nước mà policy cho prior gần nhau.

5.4. So sánh Minimax vs MCTS

	Minimax + Alpha-Beta	MCTS (PUCT)
Dùng value head?	Không (eval tính material+PST)	Có (mạng đánh giá leaf)
Quyết định	Duyệt tới depth cố định	Phân bổ simulation cho nhánh hứa hẹn
Tốc độ	Nhanh, không cần GPU	Chậm hơn (mỗi sim 1 lần gọi mạng)
Phụ thuộc chất lượng mạng	Chỉ policy (để sắp xếp)	Cả policy (P) lẫn value (Q)
Kiểm chứng	Tìm đúng chiếu hết 1 nước (test)	Chạy hợp lệ trong self-play/pit

Thực tế ở quy mô dự án: Minimax depth 3 cho nước chơi ổn định, độc lập với chất lượng value head. MCTS chỉ thực sự vượt trội khi value head đủ tốt — mà mức MAE 0,63 hiện tại chưa đảm bảo. Khuyến nghị demo bằng Minimax, còn MCTS dùng để chạy self-play.

6. Self-play loop — cơ chế, vấn đề, cách khắc phục, và kết quả

6.0. Cơ chế vòng lặp (pipeline self-play)

Mỗi iteration gồm 3 pha (vòng lặp AlphaZero thu nhỏ):

1. **Sinh dữ liệu (self-play).** Model hiện tại tự đánh `num_games` ván bằng MCTS. Mỗi nước lấy **phân bố số lần thăm (visit counts)** ở root làm **policy target** (mềm). `temperature_threshold` nước đầu dùng `temperature = 1` (chọn theo xác suất → khám phá), sau đó `= 0` (chọn nước thăm nhiều nhất). Root có Dirichlet noise. Value target = kết quả ván (đổi dấu theo bên đi). `max_moves` giới hạn độ dài ván.
2. **Retrain candidate.** Copy `current` → `candidate`, train `train_epochs` epoch trên dữ liệu vừa sinh. Loss = cross-entropy với target mềm + MSE value.
3. **Pit & chấp nhận.** `candidate` đấu `current` `pit_games` ván. Nếu **tỉ lệ thắng trong các ván phân thắng bại $\geq 55\%$** thì thay `current = candidate` (theo AlphaZero, bỏ ván hòa để giảm nhiễu). Lưu `iter_NNN.pt` + cập nhật ELO.

6.1. Vấn đề ở lần chạy đầu

Lần chạy đầu chạy đúng kỹ thuật nhưng **gần như mọi ván đều hòa** (10/10 hòa ở 4/5 vòng) → pit không phân biệt được model, accept/reject gần như ngẫu nhiên. Nguyên nhân: ván chạm `max_moves = 200` mà chưa chiếu hết bị mặc định tính **hòa**; model chơi tất định → ván kéo dài rồi bị cắt → toàn hòa.

6.2. Các bản vá đã triển khai

1. **Adjudication theo material** (`evaluation.adjudicate_result`). Khi ván chạm `max_moves`, chấm theo material+PST: bên hơn rõ (`|score| > margin`, mặc định 100cp) được tính thắng. Hòa tự nhiên vẫn là hòa. Dùng ở cả `self_play.py` và `pit.py`.
2. **ELO tracking** (`elo.py`). Chênh lệch ELO của candidate suy từ tỉ số thắng: `elo_diff = -400 \cdot \log_{10}(1/\text{score} - 1)` với `score = (\text{thắng} + 0.5 \cdot \text{hòa}) / \text{số ván}`; cộng dồn (chỉ khi accept) → đường ELO tương đối (model đầu = 0). Lưu vào `iter_*.pt`, vẽ bằng `plot_history.py`. **Lưu ý đánh giá:** ELO tương đối này CHỈ dùng để *theo dõi tiến trình self-play giữa các vòng* (model có leo thang không), **không phải thước đo sức cờ tuyệt đối** và không nên dùng làm căn cứ đánh giá model — thước đo sức cờ là benchmark với Stockfish (Mục 7).
3. **Batch inference cho MCTS** (`mcts.py`, `num_parallel`). Mặc định 1 = như cũ; >1 bật batch.

Adjudication và ELO đã được **kiểm thử đơn vị** (12/12 pass, python-chess thuần).

6.3. Kết quả sau khi vá (chạy lại 5 iteration)

```
iter1: candidate 0-10 current, 0 hòa -> reject (ELO kept 0)
iter2: candidate 0-10 current, 0 hòa -> reject (ELO kept 0)
iter3: candidate 0-10 current, 0 hòa -> reject (ELO kept 0)
iter4: candidate 5-5 current, 0 hòa -> reject (ELO kept 0)
iter5: candidate 5-5 current, 0 hòa -> reject (ELO kept 0)
```

Hai kết luận: (1) **Bản vá adjudication hoạt động** — 0 ván hòa ở mọi vòng (trước là 10/10 hòa); ELO tracking chạy đúng. (2) **Self-play KHÔNG cải thiện được model**. Candidate hoặc tệ hơn hẳn (0-10) hoặc ngang ngửa (5-5, do bên đi trước quyết định), không bao giờ $\geq 55\%$ → không accept, ELO phẳng ở 0.

Vì sao? `current` được train trên 1,77 triệu thế chuyên gia; mỗi vòng lại train 2 epoch trên chỉ ~2.400 mẫu self-play (sinh bởi MCTS yếu) ở lr $1e-3$ → **ghi đề** trọng số tốt, candidate đi nước kém. Bài học: **20 ván × 100 sim/vòng là quá ít để cải thiện một model đã pretrain mạnh**; muốn ELO leo cần lr thấp hơn ($\sim 1e-4$), replay buffer, nhiều ván/sim hơn (rất chậm trên CPU). Biểu đồ: `reports/selfplay.png`.

7. Benchmark vs Stockfish

Đây là **thước đo sức cờ chính** của dự án — đối thủ ngoài, khách quan; còn ELO tương đối ở Mục 6 chỉ để theo dõi tiến trình self-play nội bộ. Đấu 10 ván với Stockfish giới hạn **depth-2** (yếu hết mức), bot dùng MCTS 200 simulation trên `best.pt`. Mỗi bên cầm Trắng/Đen phân nửa số ván. Thời gian ~258s, ~95 nước/ván.

Kết quả	Số ván
Bot thắng	0
Hòa	6
Stockfish thắng	4
Điểm bot	3,0 / 10 (30%)

ELO ước lượng từ kết quả này (rất thô, ánh xạ depth→ELO không chuẩn): Stockfish depth-2 ≈ 1500 → bot ≈ 1353 ; chỉ nên xem là tham khảo định tính. Quan trọng hơn con số: bot hòa 6/10 nhưng không thắng ván nào, nhiều ván hòa rất dài (79–127 nước, một ván chạm 200) → chơi đủ chắc để không thua nhanh nhưng **không biết chuyển hóa lợi thế thành thắng** — đúng với điểm yếu value head (Mục 4.3, 5.3). *Để đánh giá vững hơn nên chạy thêm 20–40 ván ở depth 1/2/3 (xem Mục 11).*

8. Kiểm thử & xác minh

Bộ test pytest gồm **36 test trên 7 file**:

File	Số test	Kiểm tra gì
test_encode_board.py	5	shape 12×8×8, đếm 32 quân, encode sau 1.e4, perspective flip, wrapper tensor.
test_action_space.py	5	NUM_ACTIONS=4096, roundtrip move↔index, phong cấp, legal mask=20 nước đầu, index nhập thành.
test_network.py	3	shape output (4096 + scalar), wrapper PolicyNet, backward (có gradient).
test_search.py	4	eval thế đầu = 0, eval Trắng hơn quân, minimax tìm đúng chiếu hết 1 nước , MCTS trả nước hợp lệ.
test_pipeline.py	3	smoke end-to-end (encode→forward→MCTS→push), minimax-không-model, lưu/ nạp checkpoint.
test_adjudication_elo.py	12	adjudication (hòa/thắng/thua/checkmate/margin), ELO (đối xứng, monotonic, clamp, accept/reject, toàn hòa→0).
test_opening_book.py	4	nước đầu hợp lệ & có trong sách, đi đúng một line, trả None khi ra khỏi sách, khớp tiền tố.

Phần logic **không cần torch** (adjudication, ELO, opening book, action space) = **21 test, đã chạy và pass 21/21** trong môi trường chỉ có python-chess. Phần cần torch (encode, network, pipeline, search) chạy bằng `pytest tests/` trên máy có cài torch. Ngoài unit test, các bản vá còn được xác minh end-to-end bằng chính lần chạy self-play (0 hòa) và benchmark Stockfish (Mục 6.3, 7).

9. Quyết định thiết kế & đánh đổi

Quyết định	Lý do	Đánh đổi
Input 12 kênh (bỏ state phụ)	Đơn giản, đủ thông tin chính	Bỏ qua nhập thành / bắt tốt qua đường / lặp nước
Perspective flip khi Đen đi	Tận dụng đối xứng, ít data hơn	Phải nhớ quy ước khi debug
Action space 4096 (from×to)	Gọn, phủ hầu hết nước	Mất thông tin phong cấp dưới hậu (hiếm)
PolicyValueNet 2 đầu	AlphaZero-style, chia sẻ trunk	Value head nhẹ → khó học tốt; trunk chung có thể gây trade-off policy↔value (Mục 4.3)
Negamax + Alpha-Beta	Code gọn, cắt nhánh hiệu quả	—
Dirichlet noise tại root MCTS	Tăng khám phá khi self-play	Chỉ dùng lúc train, không lúc chơi
Ngưỡng pit 55% (ván phân thắng bại)	Theo paper AlphaZero	Nhạy với số ván nhỏ
Adjudication theo material	Phá thế "toàn hòa"	Material không phản ánh hết thế cờ

10. Bài học rút ra

- **Dữ liệu quan trọng hơn epoch.** Tăng 2k→20k ván cải thiện accuracy nhiều hơn hẳn train thêm epoch; train lâu trên data nhỏ chỉ overfit.

- **Value head là nút thắt.** Học policy (bắt chước nước) dễ hơn nhiều học value (đoán kết quả). Chất lượng MCTS bị giới hạn bởi value head (qua số hạng Q trong PUCT, Mục 5.3).
- **Self-play cần kết quả "phân thắng bại"** và phải đủ lớn. Ít ván + lr cao thì retrain làm hỏng model pretrain thay vì cải thiện.
- **Tốc độ MCTS là rào cản thực tế.** MCTS thuần Python, mỗi sim một lần gọi mạng → rất chậm; batch inference là tối ưu cần thiết.

11. Hạn chế & hướng phát triển

Hạn chế. Value head còn yếu (MAE ~0,63) → bot không chuyển hóa được lợi thế (thắng 0 ván vs Stockfish, chỉ hòa); self-play (sau khi vá) đã chạy phân thắng bại nhưng **chưa cải thiện** model do dữ liệu self-play quá nhỏ + lr quá cao; chưa làm ablation để khẳng định giả thuyết trade-off policy↔value (Mục 4.3); action space mất thông tin phong cấp dưới hậu; input 12 kênh bỏ qua trạng thái nhập thành/bắt tốt qua đường/lấp nước; benchmark Stockfish mới 10 ván ở 1 mức depth.

Hướng phát triển. (1) **Ablation** `value_weight ∈ {0, 0.5, 1, 2}` để kiểm chứng trade-off policy↔value; (2) tăng dữ liệu/độ sâu mạng để nâng value head — chìa khóa để bot biết thắng; (3) để self-play thực sự cải thiện: hạ lr retrain ~1e-4, thêm replay buffer, tăng số ván/simulation; (4) **mở rộng benchmark Stockfish** (20–40 ván ở depth 1/2/3) để có thước đo sức cờ ổn định hơn; (5) bật batch inference (`--num-parallel`) cho self-play nhanh hơn trên GPU.

12. Phụ lục

12.1. Môi trường & phụ thuộc

Python 3.10; thư viện chính (xem `requirements.txt`): `torch>=2.0`, `python-chess (1.11.x)`, `numpy`, `tqdm`, `matplotlib`, `tensorboard`, `zstandard`, `flask>=3.0`, `pytest`. Huấn luyện thực hiện trên **CPU** (~13,6h cho 10 epoch / 1,77M mẫu); `app.py` và Minimax chạy được cả khi không có torch (fallback eval tĩnh).

12.2. Bảng siêu tham số

Tham số	Giá trị	Ghi chú
channels / residual blocks	128 / 3	~1,43M params
supervised lr / optimizer	1e-3 / AdamW	+ cosine schedule, weight_decay 1e-4
value_weight	1.0	cân bằng policy/value loss
batch size	256	
grad clip	1.0	max_norm
MCTS c_puct	1.5	
MCTS simulations	100 (self-play) / 200 (chơi)	đánh đổi chất lượng/tốc độ

Tham số	Giá trị	Ghi chú
Dirichlet α / ϵ	0.3 / 0.25	noise tại root
temperature_threshold	15	số nước đầu temp=1
max_moves	200	giới hạn ván self-play/pit
accept threshold	0.55	ván phân thắng bại
adjudication margin	100 cp	chấm thắng khi cắt ván

12.3. Cách tái lập

```
# 1. Parse dữ liệu
python scripts/parse_pgn.py --input data/raw/lichess_elite_2024-01.pgn \
  --output data/processed/train.pt --max-games 20000 --min-rating 1800 --min-moves 10
# 2. Train supervised
python scripts/train_supervised.py --data data/processed/train.pt \
  --epochs 10 --batch-size 256 --channels 128 --n-res 3 --output-dir models
# 3. Self-play (adjudication + ELO; batch inference tren GPU)
python scripts/run_self_play.py --initial-model models/best.pt \
  --iterations 5 --games 20 --simulations 100 --train-epochs 2 \
  --pit-games 10 --adjudication-margin 100 --num-parallel 8 --output-dir models/selfplay
# 4. Ve biểu đồ (gồm đường ELO)
python scripts/plot_history.py --ckpt models/best.pt --output reports/supervised.png
python scripts/plot_history.py --selfplay-dir models/selfplay --output reports/selfplay.png
# 5. Benchmark vs Stockfish (can binary Stockfish)
python scripts/benchmark_stockfish.py --model models/best.pt --stockfish stockfish.exe \
  --games 10 --stockfish-depth 2 --search mcts --simulations 200
# 6. Test
pytest tests/ -v
```

12.4. Hai cách demo chơi với bot

Web app (Flask): `pip install flask` rồi `python app.py --model models/best.pt` → mở `http://localhost:5000`. Options: `--search {minimax,mcts}`, `--depth`, `--simulations`, `--no-book`, `--host`, `--port`.

Terminal (CLI): `python scripts/play_cli.py --model models/best.pt --search minimax --depth 3` (thêm `--color black`, `--opening-book`, hoặc `--search mcts --simulations 200`). Trong game nhập nước đi UCI (e2e4, g1f3, e7e8q); lệnh `undo` / `fen` / `quit`.

13. Tài liệu tham khảo

1. Silver et al. (2017), *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm* (AlphaZero) — [arXiv:1712.01815](https://arxiv.org/abs/1712.01815).
2. python-chess — <https://python-chess.readthedocs.io/>
3. Lichess open database — <https://database.lichess.org/>
4. Lichess Elite database (đã lọc rating) — <https://database.nikonoel.fr/>
5. Chess Programming Wiki (Negamax, Alpha-Beta, Piece-Square Tables) — <https://www.chessprogramming.org/>
6. Stockfish — <https://stockfishchess.org/>